Ejercicio 1 – Encontrar el mínimo de un vector

Prompt redactado:

Instrucción para OpenCode:

1. Estrategia y Justificación: Antes de escribir el código, explica la estrategia para encontrar el valor mínimo en un vector de enteros desordenado. La estrategia correcta es un recorrido lineal de única pasada, ya que necesitamos revisar cada elemento al menos una vez para garantizar que encontramos el mínimo absoluto. La complejidad temporal esperada es $O(n)$ y la complejidad espacial es $O(1)$.
2. Código en Java: Implementa el algoritmo en Java. El código debe estar exhaustivamente comentado.
3. Explicación requerida: Incluye una sección en los comentarios o en la salida explicando por qué NO es necesario ordenar el vector antes de buscar el mínimo (destacando que ordenar tomaría $O(n \log n)$, lo cual empeoraría la eficiencia).

Ejercicio 2 – Buscar un elemento en un vector desordenado

Prompt redactado:

Instrucción para OpenCode:

1. Estrategia e Implementación: Escribe una función en Java que busque un elemento en un vector desordenado utilizando la estrategia de búsqueda lineal (la única viable sin orden previo).
2. Contador de Recorrido: Haz que el programa cuente e informe en consola la cantidad exacta de posiciones/elementos que fueron recorridos hasta encontrar el elemento o determinar que no existe.
3. Análisis de Complejidad: Incluye en la respuesta el análisis formal de:
    - Mejor caso: $O(1)$ (si el elemento está en la primera posición).
    - Peor caso: $O(n)$ (si el elemento está al final o no existe).
    - Caso promedio: $O(n)$ (en promedio se recorren $n/2$ posiciones).

Ejercicio 3 – Buscar un elemento en un vector ordenado

Prompt redactado:

Instrucción para OpenCode:

1. Explicación teórica: Explica por qué la Búsqueda Binaria (dividir el espacio de búsqueda a la mitad en cada paso) es significativamente más eficiente

que la búsqueda lineal cuando los datos están previamente ordenados. Indica que la complejidad temporal esperada es $O(\log n)$.

2. Código en Java: Implementa el algoritmo de Búsqueda Binaria en Java.
3. Seguimiento paso a paso: Asegúrate de que el código imprima en consola la evolución de las variables principales (inicio, fin, medio y el valor del elemento central) en cada iteración del bucle.

Ejercicio 4 – Detectar elementos duplicados

Prompt redactado:

Instrucción para OpenCode:

Proporciona la implementación en Java de dos soluciones distintas para detectar si existen elementos duplicados dentro de un vector de enteros:

1. Solución 1: Utilizando dos ciclos for anidados.
2. Solución 2: Utilizando una estructura de datos HashSet.

Al finalizar el código, incluye una tabla o sección comparativa analizando ambas soluciones respecto a su complejidad temporal ($O(n^2)$ vs $O(n)$) y su complejidad espacial ($O(1)$ vs $O(n)$ auxiliar).

Ejercicio 5 – Contar ocurrencias

Prompt redactado:

Instrucción para OpenCode:

1. Código en Java: Implementa un algoritmo en Java que cuente e informe cuántas veces aparece un determinado valor dentro de un vector de enteros.
2. Justificación: Explica en la respuesta por qué el algoritmo está obligado a recorrer el vector de principio a fin de manera completa sin cortar antes la ejecución.
3. Análisis de Complejidad: Indica la complejidad temporal ($O(n)$) y espacial ($O(1)$) del algoritmo en el mejor, peor y promedio de los casos.

Ejercicio 6 – Comparar dos vectores

Prompt redactado:

Instrucción para OpenCode:

1. Código en Java: Implementa un algoritmo que compare si dos vectores son exactamente iguales (mismo tamaño y mismos elementos en las mismas posiciones).

2. Optimización de corte: El algoritmo debe finalizar la ejecución mediante un retorno temprano o break apenas detecte la primera diferencia entre ambos vectores (o si sus tamaños difieren).
3. Análisis de Complejidad: Detalla el análisis de:
    - Mejor caso: $O(1)$ (difieren en la primera posición o en tamaño).
    - Peor caso: $O(n)$ (son idénticos o la diferencia está en la última posición).
    - Caso promedio: $O(n)$.

Ejercicio 7 – Invertir un vector

Prompt redactado:

Instrucción para OpenCode:

Implementa en Java dos versiones distintas para invertir un vector de enteros:

1. Versión A (Vector auxiliar): Crea un segundo vector del mismo tamaño para copiar los elementos en orden inverso.
2. Versión B (In-place / Dos punteros): Invierte el vector original intercambiando los elementos de los extremos hacia el centro sin usar memoria adicional significativa.

Incluye una comparación detallada entre ambas alternativas evaluando ventajas, desventajas, complejidad temporal ($O(n)$ en ambas) y complejidad espacial ($O(n)$ vs $O(1)$).

Ejercicio 8 – Sumar todos los elementos de una matriz

Prompt redactado:

Instrucción para OpenCode:

1. Código en Java: Implementa un programa que calcule la suma total de los elementos de una matriz bidimensional (recorriéndola por filas y columnas mediante ciclos anidados).
2. Métrica de Operaciones: Añade una variable contador que registre e imprima por pantalla la cantidad exacta de operaciones de suma o iteraciones ejecutadas.
3. Análisis de Complejidad: Explica por qué la complejidad temporal es $O(N \times M)$ (donde N es filas y M columnas, o $O(n^2)$ para una matriz cuadrada de $n \times n$).

Ejercicio 9 – Encontrar el mayor elemento de una matriz

Prompt redactado:

Instrucción para OpenCode:

1. Código en Java: Implementa un algoritmo para encontrar el elemento de valor máximo en una matriz bidimensional.
2. Justificación Teórica: Explica detenidamente por qué, al no estar la matriz ordenada de forma particular, es estrictamente necesario examinar todos y cada uno de los elementos.
3. Análisis de Complejidad: Especifica la complejidad temporal y espacial esperada para matrices de $\mathrm{N} \times \mathrm{M}$.

Ejercicio 10 – Ordenamiento burbuja (Bubble Sort)

Prompt redactado:

Instrucción para OpenCode:

1. Explicación didáctica: Explica por qué el algoritmo Bubble Sort es considerado adecuado casi exclusivamente con fines didácticos o para conjuntos de datos muy pequeños, debido a su ineficiencia $\mathrm{O}(\mathrm{n}^2)$.
2. Código en Java: Implementa el algoritmo de ordenamiento por burbuja en Java.
3. Contadores de rendimiento: Modifica el programa para que cuente y muestre al finalizar la cantidad total de comparaciones realizadas y la cantidad total de intercambios (swaps) ejecutados durante el proceso.